

Reflexion Agent — Explained Line by Line

A plain-language walkthrough of chains.py, schemas.py, tool_executor.py and main.py

What are we actually building?

This code builds a **self-correcting research agent**, using a design pattern called **Reflexion**. Instead of just asking an AI model a question once and taking whatever it says, this system makes the AI answer, then criticize its own answer, then search the web to fill the gaps it found, and then rewrite a better answer using that new information. This can repeat a few times, so the final answer is more accurate and better researched than a single one-shot response.

Think of it like a student writing an essay: write a first draft, have a strict teacher point out what's missing or unnecessary, go look up the missing facts in a library, then rewrite the essay with proper citations. This code automates that whole loop using an LLM (a Groq-hosted open model) as both the writer and the critic, and Tavily (a web search API) as the library.

How the four files fit together

The project is built with **LangChain** (for talking to the LLM and structuring prompts) and **LangGraph** (for wiring together a repeatable loop / state machine of steps, called a 'graph'). Each file has one job:

File	Role in the system
schemas.py	Defines the exact 'shape' of data the AI must return: an answer, a self-critique, and follow-up search queries.
chains.py	Builds two ready-to-use pipelines ('chains'): one that writes the first draft answer, and one that revises it later
tool_executor.py	Takes the search queries the AI generated and actually runs them against the Tavily web search API, then tri
main.py	Wires everything into a loop using LangGraph: draft -> search -> revise -> (search -> revise again, up to a lim

The flow, step by step

1. **draft** — first_responder (from chains.py) writes a first answer + self-critique + search queries, forced into the AnswerQuestion shape.
2. **execute_tools** — tool_executor.py takes those search queries and actually searches the web with Tavily.
3. **revise** — revisor (from chains.py) reads the search results and rewrites a better, cited answer, forced into the ReviseAnswer shape.
4. LangGraph checks how many times tools have been called. If under the limit, it loops back to execute_tools with the new queries from the revision. If the limit is hit, it stops and returns the final message.

Every explanation below goes file by file, and inside each file, section by section — in plain language, not textbook definitions.

schemas.py

This file doesn't run any logic — it only defines the 'shape' of the data the AI must produce at each stage. Everything else in the project imports from here.

```
from typing import List

from pydantic import BaseModel, Field
```

We import **List** just so we can say 'this field is a list of strings' further down. **Pydantic** is a library that lets you define a class that describes exactly what fields a piece of data must have, and what type each field is (text, number, list, etc). We use it here for a very specific reason: later we will force the AI model to fill in one of these classes instead of replying with plain paragraphs. That gives us structured, predictable output we can read in code, instead of trying to parse messy free-form text.

```
class Reflection(BaseModel):
    missing: str = Field(description="Critique of what is missing.")
    superfluous: str = Field(description="Critique of what is superfluous")
```

This defines a small structure called **Reflection** — basically the AI's self-review of its own answer. It has two text fields: **missing** (what the answer left out) and **superfluous** (what the answer included that it didn't need to). The 'description' text inside Field(...) isn't just a comment — it's actually sent to the AI model as an instruction, telling it what kind of content belongs in that field.

```
class AnswerQuestion(BaseModel):
    """Answer the question."""

    answer: str = Field(description="250 word detailed answer to the question.")
    reflection: Reflection = Field(description="Your reflection on the initial answer.")
    search_queries: List[str] = Field(
        description="1-3 search queries for researching improvements to address the critique of your current answer."
    )
```

This is the structure the AI must fill in for its **first draft**. It bundles three things together in one response: the actual **answer** text, a **reflection** (using the Reflection class above) critiquing that answer, and a list of 1-3 **search_queries** the AI thinks would help it improve the answer. The triple-quoted text right under the class name ('Answer the question.') is a docstring, and LangChain actually uses this as the description of the 'tool' when it tells the AI model what tools/functions are available to call — so it's not just documentation for humans, it's read by the AI too.

```
class ReviseAnswer(AnswerQuestion):
    """Revise your original answer to your question."""

    references: List[str] = Field(
        description="Citations motivating your updated answer."
    )
```

ReviseAnswer **inherits** everything from AnswerQuestion (answer, reflection, search_queries) and adds one more field: **references**, a list of URLs backing up the claims in the revised answer. This is the structure the AI must fill in during the revision step, after it has seen search results. Reusing AnswerQuestion via inheritance avoids repeating the same three fields twice — it just adds what's new for this second stage.

chains.py

This file sets up the AI model connection and builds the two prompt-to-LLM pipelines ('chains') that do the actual thinking: one for the first draft, one for revisions.

```
import datetime

from dotenv import load_dotenv

load_dotenv()
```

datetime is Python's built-in module for working with dates and times — used later to tell the AI what the current time is. **dotenv** is a small library that reads a .env file (a plain text file with lines like GROQ_API_KEY=xxxx) and loads those values as environment variables. **load_dotenv()** actually runs that loading step, so that when we create the Groq client a few lines later, it can automatically find the API key without us typing it into the code.

```
from langchain_core.messages import HumanMessage
from langchain_core.output_parsers.openai_tools import (
    JsonOutputKeyToolsParser, PydanticToolsParser)
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_groq import ChatGroq
```

HumanMessage is just a wrapper LangChain uses to represent 'this text came from the user', used later in the test block at the bottom of the file. **JsonOutputKeyToolsParser** and **PydanticToolsParser** are two different ways to take the AI's raw tool-call response and turn it into something usable in Python — one gives you plain JSON, the other gives you an actual AnswerQuestion object. **ChatPromptTemplate** lets us build a reusable prompt with blanks that get filled in later (like a mail-merge template for prompts), and **MessagesPlaceholder** is a special blank inside that template that gets replaced with the whole running conversation history. **ChatGroq** is the LangChain wrapper that lets us talk to an AI model hosted on Groq's fast inference servers.

```
from schemas import AnswerQuestion, ReviseAnswer
```

This pulls in the two data 'shapes' we defined in schemas.py, so we can tell the AI model 'you must reply using exactly this structure'.

```
llm = ChatGroq(model="openai/gpt-oss-120b", temperature=0, max_tokens=4096)
```

This line creates the actual AI model client we'll use everywhere in this project. **model** picks which specific model to call (an open-weight 120B-parameter model served on Groq). **temperature=0** tells it to be as deterministic and non-random as possible — good for research tasks where you want consistent, focused answers rather than creative variety. **max_tokens=4096** caps how long a single response is allowed to be, as a safety limit.

```
parser = JsonOutputKeyToolsParser(key_name="AnswerQuestion", return_id=True)
parser_pydantic = PydanticToolsParser(tools=[AnswerQuestion])
```

These create two optional 'translators' for the AI's tool-call output. **parser** would pull out just the AnswerQuestion part of the response as plain JSON, keeping the tool-call id too. **parser_pydantic** would convert the response straight into a real AnswerQuestion Python object. Note: neither of these is actually used in the main graph (main.py) — they're leftovers/utilities, only parser_pydantic gets used in the manual test at the bottom of this file.

```
actor_prompt_template = ChatPromptTemplate.from_messages(
    [
        (
```

```

        "system",
        """You are expert researcher.
Current time: {time}

1. {first_instruction}
2. Reflect and critique your answer. Be severe to maximize improvement.
3. Recommend search queries to research information and improve your answer.

You MUST respond only by calling the provided tool/function with these fields filled in.
Do NOT write your answer as plain text or markdown. Do NOT respond outside the tool call.""",
    ),
    MessagesPlaceholder(variable_name="messages"),
]
).partial(time=lambda: datetime.datetime.now().isoformat())

```

This builds the base instruction ('system prompt') given to the AI on every call. It sets the AI's role ('expert researcher'), inserts the current time, gives it a 3-step job list, and then strongly instructs it to reply *only* through the structured tool call (from schemas.py) and never as plain text — this is what makes the structured-output trick reliable. **{time}** and **{first_instruction}** are placeholders that get filled in later.

MessagesPlaceholder(variable_name="messages") means 'insert the entire conversation so far here' when this template is actually used. **.partial(time=lambda: ...)** pre-fills the {time} blank automatically every time the template is used, by calling that function fresh each time, so the timestamp is always current instead of frozen at import time.

```

first_responder_prompt_template = actor_prompt_template.partial(
    first_instruction="Provide a detailed 250 word answer."
)

```

This takes the shared template above and fills in the remaining {first_instruction} blank with 'write a 250 word answer' — this version of the template is specifically for producing the **first draft**.

```

first_responder = first_responder_prompt_template | llm.bind_tools(
    tools=[AnswerQuestion], tool_choice="AnswerQuestion"
).with_retry(stop_after_attempt=5)

```

This builds the actual runnable pipeline (a 'chain') used to produce the first draft. The **|** symbol is LangChain's way of piping one step's output into the next step's input — here, the filled-in prompt goes straight into the LLM. **llm.bind_tools(tools=[AnswerQuestion], tool_choice="AnswerQuestion")** tells the model 'the only tool you're allowed to call is AnswerQuestion, and you must call it' — this is the actual mechanism that forces structured output matching our schema. **.with_retry(stop_after_attempt=5)** automatically retries the call up to 5 times if it fails (e.g. due to a network hiccup or the model returning something malformed), instead of crashing the whole program on the first failure.

```

revise_instructions = """Revise your previous answer using the new information.
- You should use the previous critique to add important information to your answer.
- You MUST include numerical citations in your revised answer text so it can be verified, e.g. [1], [2].
- Do NOT write a "References" section inside the answer text.
- Instead, populate the separate `references` field of the tool call with the full URL for each reference.
- You should use the previous critique to remove superfluous information from your answer and make sure it is under 250 words.
"""

```

This is a plain text instruction string used for the **revision** stage. It tells the AI to use the search results and its own earlier critique to improve the answer, to add numbered citations like [1], [2] inside the text, to put the actual URLs in the separate 'references' field (not typed out inside the answer itself), and to keep the whole thing under 250 words even after adding new information.

```

revisor = actor_prompt_template.partial(
    first_instruction=revise_instructions
) | llm.bind_tools(tools=[ReviseAnswer], tool_choice="ReviseAnswer").with_retry(stop_after_attempt=5)

```

Same pattern as `first_responder`, but for revision: fill the shared template's blank with the `revise_instructions` text, then force the model to reply using the **ReviseAnswer** shape (which includes the extra 'references' field) instead of plain `AnswerQuestion`. This is the chain that produces every improved answer after the first draft.

```

if __name__ == "__main__":
    human_message = HumanMessage(
        content="Write about AI-Powered SOC / autonomous SOC problem domain,"
        " list startups that do that and raised capital."
    )

    chain = (
        first_responder_prompt_template
        | llm.bind_tools(tools=[AnswerQuestion], tool_choice="AnswerQuestion")
        | parser_pydantic
    )

    res = chain.invoke(input={"messages": [human_message]})
    print(res)

```

This block only runs if you execute `chains.py` directly (not when it's imported by another file — that's what `'if __name__ == "__main__"'` means in Python). It's a standalone manual test: it builds one `HumanMessage` with a sample research question, builds a one-off chain that also converts the output into a real `AnswerQuestion` Python object using `parser_pydantic`, runs it, and prints the result — useful for quickly checking that the prompt and model are working correctly before plugging them into the full LangGraph pipeline in `main.py`.

tool_executor.py

This file is the 'hands' of the agent — it takes the search queries the AI thinks up and actually goes and searches the web for them, then trims the results down to a safe size.

```
from dotenv import load_dotenv

load_dotenv()
```

Same as in chains.py — loads API keys (here, the Tavily search API key) from a .env file into environment variables so the TavilySearch client below can find them automatically.

```
from langchain_core.tools import StructuredTool
from langchain_tavily import TavilySearch
from langgraph.prebuilt import ToolNode

from schemas import AnswerQuestion, ReviseAnswer
```

StructuredTool is LangChain's way of wrapping a normal Python function so it can be called as a 'tool' inside a LangGraph graph, with a defined name. **TavilySearch** is the client for Tavily, a search API built specifically for AI agents (it returns clean, summarized web results instead of raw HTML). **ToolNode** is a ready-made LangGraph building block whose whole job is: look at the last AI message, see what tool(s) it asked to call, run them, and turn the results back into messages the graph can continue with. We import AnswerQuestion and ReviseAnswer again here just to reuse their class names as tool names, keeping everything consistent.

```
tavily_tool = TavilySearch(max_results=3)
```

Creates one reusable Tavily search client, configured to return at most 3 results per query — keeping things fast and limiting how much text comes back.

```
def run_queries(search_queries: list[str], **kwargs):
    """Run the generated queries."""
    results = tavily_tool.batch([{"queries": query} for query in search_queries])
    # trim content to keep context small enough for free-tier TPM limits
    for r in results:
        if isinstance(r, dict) and "results" in r:
            for item in r["results"]:
                if "content" in item and item["content"]:
                    item["content"] = item["content"][:500]
    return results
```

This is the actual function that gets called whenever the AI has produced a list of search_queries. It takes that list of query strings and runs all of them through Tavily using **.batch(...)**, which sends multiple searches out and waits for all of them, instead of doing them one at a time. ****kwargs** just means 'accept and ignore any other fields the tool call might include' (like reflection or answer), so the function doesn't crash if extra data gets passed in. The loop after that goes through every search result and, if it has a 'content' field, **cuts it down to the first 500 characters**. This matters practically: Groq's free tier has a limit on tokens-per-minute, and raw search result text can be long, so trimming it keeps the agent from hitting that limit and failing.

```
execute_tools = ToolNode(
    [
        StructuredTool.from_function(run_queries, name=AnswerQuestion.__name__),
        StructuredTool.from_function(run_queries, name=ReviseAnswer.__name__),
    ]
)
```

)

This creates the actual LangGraph node used in main.py. It registers the *same* run_queries function under **two different tool names**: "AnswerQuestion" and "ReviseAnswer" (AnswerQuestion.__name__ just means 'the string name of that class', so this evaluates to the literal text "AnswerQuestion" and "ReviseAnswer"). This is necessary because the draft step produces a tool call named AnswerQuestion and the revise step produces one named ReviseAnswer — registering both names under the same underlying function means ToolNode can correctly handle tool calls coming from either stage of the loop, using one shared piece of logic.

main.py

This is the file you actually run. It wires `draft_node`, `execute_tools`, and `revise_node` together into a loop using `LangGraph`, then runs that loop end to end on a real question.

```
from typing import Literal

from langchain_core.messages import AIMessage, ToolMessage
from langgraph.graph import END, START, StateGraph, MessagesState
```

Literal is used purely for type-hinting — to tell Python/your editor 'this function can only return one of these exact string values'. **AIMessage** represents a reply that came from the AI model, and **ToolMessage** represents the result of running a tool — both are used to check what kind of message we're looking at further down.

StateGraph is `LangGraph`'s main class for building a graph of steps ('nodes') that pass a shared piece of state between them. **START** and **END** are special fixed markers for 'where the graph begins' and 'where the graph should stop'. **MessagesState** is a ready-made state format that's just 'a list of messages' — perfect for a chat-style agent where every step adds to the same conversation.

```
from chains import revisor, first_responder
from tool_executor import execute_tools
```

Pulls in the three main pieces we built in the other files: the chain that writes the first draft, the chain that revises it, and the node that actually runs web searches.

```
MAX_ITERATION = 2
```

A simple safety limit: the agent will not be allowed to loop more than this many times. Without a cap like this, the draft -> search -> revise loop could in theory keep going forever.

```
def draft_node(state: MessagesState):
    """Draft the initial response."""
    response = first_responder.invoke({"messages": state["messages"]})
    return {"messages": [response]}
```

This defines the very first step of the graph. `LangGraph` will call this function and hand it the current **state** (which, at the start, is just the user's original question wrapped as a message). It runs the `first_responder` chain from `chains.py` on the existing messages, gets back the AI's structured first-draft response, and returns it wrapped in a dict with a 'messages' key containing a list. `LangGraph` automatically **appends** that new message onto the running conversation state rather than replacing it — that's the default behavior of `MessagesState`.

```
def revise_node(state: MessagesState):
    """Revise the answer based on """
    response = revisor.invoke({"messages": state["messages"]})
    return {"messages": [response]}
```

Same idea as `draft_node`, but it calls the `revisor` chain instead. Because it receives the whole conversation-so-far (including the earlier draft and the search results that came back from `execute_tools`), the `revisor` chain has everything it needs to write a better, cited answer. (The docstring here is left unfinished mid-sentence in the original code — a small leftover typo, it doesn't affect how the code runs.)

```
def event_loop(state: MessagesState) -> Literal["execute_tools", END]:
    """Determine whether to continue or end based on iteration count."""
    count_tool_visits = sum(
```

```

        isinstance(item, ToolMessage) for item in state["messages"]
    )
    num_iterations = count_tool_visits
    if num_iterations > MAX_ITERATION:
        return END
    return "execute_tools"

```

This function decides what happens *after* every revision: keep looping, or stop. It counts how many ToolMessage entries exist in the conversation so far — in other words, how many times search has already been run. If that count is already past MAX_ITERATION (2), it tells the graph to go to **END** and stop. Otherwise, it sends the graph back to **"execute_tools"** so the newest search_queries (from the latest revision) get searched too, and the loop continues into another revise step.

```

builder = StateGraph(MessagesState)
builder.add_node("draft", draft_node)
builder.add_node("execute_tools", execute_tools)
builder.add_node("revise", revise_node)

```

This creates the actual graph object, using MessagesState as the shape of data passed between steps, and registers each function as a named **node** in that graph: "draft", "execute_tools", and "revise". Naming them is what lets us wire them together with edges next.

```

builder.add_edge(START, "draft")
builder.add_edge("draft", "execute_tools")
builder.add_edge("execute_tools", "revise")

```

These lines draw the fixed, always-happens connections between steps: start the graph by going to "draft", after "draft" always go to "execute_tools" (run the first round of searches), and after "execute_tools" always go to "revise" (write the improved answer).

```

builder.add_conditional_edges("revise", event_loop, ["execute_tools", END])

```

This is the only **conditional** connection — after "revise" finishes, instead of always going to a fixed next step, LangGraph calls the event_loop function to decide where to go next: either back to "execute_tools" (search again using the newest queries, then revise again) or to **END** (stop and return the final result). This conditional edge is what actually creates the repeating loop and eventually breaks out of it.

```

graph = builder.compile()

```

Turns the node/edge definitions into an actual runnable graph object. Nothing has executed yet — this just finalizes the wiring.

```

print(graph.get_graph().draw_mermaid())

```

Prints out the graph's structure as Mermaid diagram syntax (a text format for drawing flowcharts) — purely for visualizing/debugging the flow, it doesn't affect how the agent runs.

```

res = graph.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": "Write about AI-Powered SOC / autonomous soc problem domain, list startups that"
            }
        ]
    }
)

```

```
    }  
)
```

This is what actually runs the whole thing. It calls the compiled graph with a starting state: one user message containing the research question. From here, LangGraph runs `draft`, then `execute_tools`, then `revise`, then keeps looping (`search`, `revise`, `search`, `revise`...) according to `event_loop`'s decision, until it hits `END`. The final result — the whole message history — is stored in **res**.

```
last_message = res["messages"][-1]  
if isinstance(last_message, AIMessage) and last_message.tool_calls:  
    print(last_message.tool_calls[0]["args"]["answer"])  
print(res)
```

After the graph finishes, this grabs the very last message in the conversation. If that message is an `AIMessage` and it contains at least one tool call (which it should, since every response is forced through `AnswerQuestion/ReviseAnswer`), it prints just the final **answer** text out of that tool call's arguments — this is the clean, human-readable final answer. Then it also prints the entire **res** object, which shows the full back-and-forth of messages, searches, and revisions for debugging.

Quick recap

1. `schemas.py` defines the required data shapes. 2. `chains.py` connects those shapes to an actual AI model via prompts, forcing structured tool-call replies. 3. `tool_executor.py` takes the AI's `search_queries` and runs real web searches, trimming results to stay within token limits. 4. `main.py` stitches `draft` -> `search` -> `revise` into a LangGraph loop with a hard cap of 2 extra iterations, then runs it and prints the final researched, cited answer.